TCS302 - DATA STRUCTURES WITH C

# Introduction to Sorting

Terminology, Stability and Efficiency

**Unit IV - Lecture 1**

**Terminology** key, record, pass, comparison, move

**Stability** do equal keys keep their original order?

**Efficiency** comparisons, moves, memory, best and worst case

# Why sort at all?

In Unit III, Lecture 7, binary search found a key in O(log n) - but only on SORTED data. Sorting is the price paid up front to make every later search cheap.

- Sort once, search many times: one O(n log n) sort can pay for thousands of O(log n) searches.
- Sorted data makes duplicates adjacent, merging two lists simple, and ranking or reporting straightforward.
- Many algorithms later in this course (merge, BST building, Kruskal's algorithm) either sort or rely on sorted order.

# Sorting vocabulary

| Term | Meaning |
|---|---|
| **Key** | The field the ordering is based on (for example, marks) |
| **Record** | A key together with its other data (name, roll number, ...) |
| **Ascending / descending** | Smallest key first / largest key first |
| **Pass** | One complete sweep of the algorithm over the data |
| **Comparison** | Asking whether one key should come before another |
| **Move** | Copying, shifting or swapping a record |
| **Duplicates** | Different records that carry equal keys |

*Every sorting algorithm is judged by how many comparisons and moves it needs.*

# Three ways to classify a sorting method

**Internal vs external**
Internal: all data fits in main memory (this unit). External: data lives on disk and is sorted in chunks (Unit V ideas).

**In-place vs extra memory**
In-place: needs only O(1) extra space beyond the array. Others (such as merge sort) need a second array of size n.

**Comparison-based or not**
Comparison sorts only ever ask 'is A before B?'. Every sort in this unit is comparison-based.

Scope of Unit IV: internal, comparison-based sorting of records held in an array (plus a binary-heap and tree variant).

# Stability: what happens to equal keys?

A sort is STABLE if records with equal keys stay in the same relative order after sorting as they had before sorting.

| Sorting four records by marks | Resulting order |
|---|---|
| **Before sorting** | `Asha(80)  Bina(75)  Charu(80)  Dev(60)` |
| **A stable sort gives** | `Dev(60)  Bina(75)  Asha(80)  Charu(80)` |
| **An unstable sort may give** | `Dev(60)  Bina(75)  Charu(80)  Asha(80)` |

- Asha and Charu both have 80. Asha came first, so a stable sort keeps Asha before Charu.
- Why it matters: sort by name first, then stably by marks, and ties in marks stay alphabetical (Unit IV, Lecture 7).

# Verified: one sort is stable, one is not

```
/* selection sort: swap the minimum
   into position i */
struct Rec t = a[i];
a[i] = a[min];
a[min] = t;

/* insertion sort: shift larger keys
   right, stop at an equal key */
while (j >= 0 &&
       a[j].marks > key.marks)
{
    a[j + 1] = a[j];
    j--;
}
```

| Method | Result (marks 80, 75, 80, 60) |
|---|---|
| Original | Asha Bina Charu Dev |
| Selection | Dev Bina Charu Asha |
| Insertion | Dev Bina Asha Charu |

- Selection: Charu is now ahead of Asha - NOT stable.
- Insertion: Asha is still ahead of Charu - stable.

Cause: the first swap sends Asha from index 0 to index 3, jumping over Charu. Insertion sort only ever shifts one step and never passes an equal key.

# Efficiency: what do we actually count?

| Measure | Why it matters |
|---|---|
| **Comparisons** | Each one costs time, especially when keys are strings or large records |
| **Moves (swaps or shifts)** | Moving big records is expensive; some sorts move far fewer than others |
| **Extra memory** | In-place sorts need O(1); others need a second array of size n |

**Best, average and worst case**
Cost depends on the INPUT as well as the algorithm. The best case (often already sorted), the worst case (often reversed) and the average case (random order) can differ enormously. Always ask which case a quoted bound refers to.

# Measured: same algorithms, three different inputs (n = 8)

| Input | Selection: comparisons / swaps | Insertion: comparisons / shifts |
|---|---|---|
| **Already sorted (1..8)** | 28 / 0 | 7 / 0 |
| **Reversed (8..1)** | 28 / 4 | 28 / 28 |
| **Mixed (5 2 8 1 9 3 7 4)** | 28 / 5 | 18 / 13 |

- Selection sort makes exactly n(n-1)/2 = 28 comparisons whatever the input - it cannot take advantage of existing order.
- Insertion sort needs only n-1 = 7 comparisons on sorted data, but the full 28 and 28 shifts on reversed data.
- Neither algorithm is simply 'better': it depends on the input you expect.

# Is there a floor? The best any comparison sort can do

n distinct keys can arrive in n! different orders. Each comparison has two outcomes, so it can at best halve the possibilities. Telling n! orders apart therefore needs at least log2(n!) comparisons in the worst case.

| n | n! possible orders | minimum worst-case comparisons | n(n-1)/2 for comparison |
|---|---|---|---|
| 4 | 24 | 5 | 6 |
| 8 | 40,320 | 16 | 28 |
| 10 | 3,628,800 | 22 | 45 |

*log2(n!) grows like n log n, so O(n log n) is the best possible for any comparison-based sort.*

# What the rest of Unit IV will measure

| Lecture | Sort | Typical time | Stable? |
|---|---|---|---|
| 2 | Insertion sort | `O(n^2), best O(n)` | yes |
| 3 | Bubble sort | `O(n^2)` | yes |
| 4 | Quick sort | `avg O(n log n), worst O(n^2)` | usually no |
| 5 | Merge sort (two-way) | `O(n log n)` | yes |
| 6 | Heap sort | `O(n log n)` | no |

For every sort, the same three questions will be asked: how many comparisons and moves, how much extra memory, and is it stable?

UNIT IV - LECTURE 1 - SUMMARY

# Judge a sort by order, cost and memory

- A key decides the order; a record is a key plus its data. A pass is one sweep; comparisons and moves are the cost units.
- Stable = equal keys keep their original order. Verified: insertion sort kept Asha before Charu, selection sort did not.
- Cost depends on input: on n = 8, insertion sort used 7 comparisons (sorted) or 28 (reversed); selection sort always used 28.
- No comparison sort can beat log2(n!) comparisons - about n log n - so O(n log n) is the target for the sorts ahead.

**EXIT TICKET** Records (key, label): 4a, 3b, 4c, 1d. Sort by key using selection sort. Is the order of 4a and 4c preserved? Then sort with insertion sort and compare.